

Universidad Nacional de Río Cuarto
Facultad de Ciencias Exactas, Físico-Químicas y Naturales
Departamento de Computación
Licenciatura en Ciencias de la Computación

Bases de Datos II

Resumen Teórico

Transacciones y Control de Concurrency
Práctico N.º 6

Alumno: Tomás Rodeghiero
Año: 2026

Índice

1. Qué es una transacción	2
2. Propiedades ACID	2
3. Estados de una transacción	2
4. Ejecución concurrente y planificaciones	3
4.1. Planificación (<i>schedule</i>)	3
5. Serializabilidad	3
5.1. Conflicto entre operaciones	4
5.2. Equivalencia por conflicto	4
5.3. Test del grafo de precedencia	4
6. Recuperabilidad	4
6.1. Planificación recuperable	4
6.2. Planificación sin cascada	4
7. Niveles de aislamiento (SQL 92)	5
8. Control de concurrencia: protocolos basados en bloqueos	5
8.1. Modos de bloqueo	5
8.2. Posibilidad de Deadlock	6
9. Protocolo de bloqueo de dos fases (2PL)	6
9.1. Variantes	6
9.2. Conversión de bloqueos	6
10. Protocolos basados en grafos	7
10.1. Protocolo de árbol	7
11. Protocolo de ordenación por marcas temporales	7
11.1. Reglas del protocolo	7
11.2. Propiedades	8
12. Esquemas multiversión y MVCC	8
12.1. Ordenación por marcas temporales multiversión	8
12.2. MVCC en PostgreSQL	8
12.3. Modos de bloqueo de tabla en PostgreSQL	9
13. Fenómeno fantasma e índices	9
14. Transacciones en SQL	9
15. Cómo se conecta con el Práctico 6	10
16. Conclusión	10

1. Qué es una transacción

Una **transacción** es una unidad lógica de trabajo de una aplicación que accede —y posiblemente modifica— elementos de la base de datos. Para el motor, una transacción es una secuencia de operaciones $r(Q)$ (lectura de un ítem Q) y $w(Q)$ (escritura) terminada por una operación de finalización: COMMIT si se completó con éxito o ROLLBACK si hubo que abortarla.

La regla básica. *La base de datos parte de un estado consistente y, después de ejecutar una transacción, debe quedar consistente otra vez.* Durante la ejecución la base puede pasar temporalmente por estados inconsistentes —por ejemplo, A ya descontó \$500 pero B todavía no los recibió—, pero esos estados son privados de la transacción: nadie de afuera debería verlos.

Ejemplo canónico. Transferir \$500 de la cuenta A a la B :

`read(A); A := A - 500; write(A); read(B); B := B + 500; write(B).`

Las garantías que se le piden a la transacción son tres:

- Si el sistema falla entre el `write(A)` y el `write(B)`, el cambio sobre A *no* debe quedar reflejado: la base quedaría inconsistente.
- Ninguna otra transacción debe ver el estado intermedio (con A ya descontado y B aún sin acreditar).
- Una vez que la transacción confirma, los \$500 están transferidos y eso se mantiene aunque después falle hardware o software.

2. Propiedades ACID

Las propiedades que un motor debe asegurar a cada transacción se resumen en el acrónimo **ACID**:

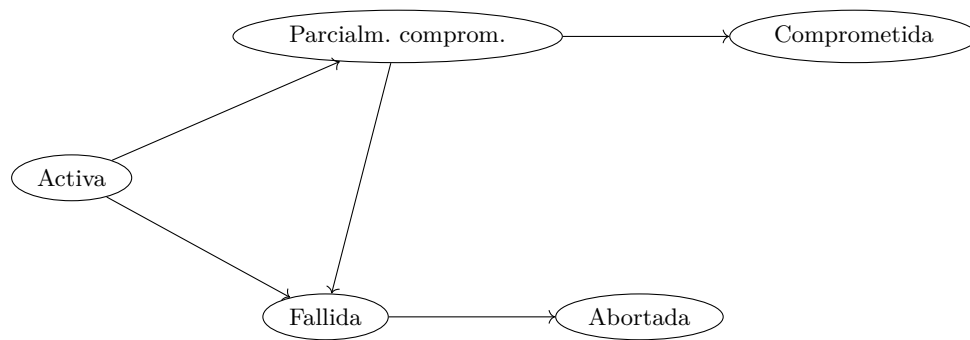
- **Atomicidad (A).** O todas las operaciones de la transacción se realizan, o ninguna. No hay “a medias”. La implementa el manejador de recuperación (logs + rollback).
- **Consistencia (C).** La ejecución aislada de una transacción *correctamente programada* preserva la consistencia de la base. Esta propiedad cae en la responsabilidad del programador: el motor no sabe qué reglas de negocio son “consistentes”.
- **Aislamiento (I).** Aunque varias transacciones se ejecuten concurrentemente, cada una percibe la base como si fuera la única que está actuando sobre ella. Es lo que implementa el control de concurrencia (bloqueos, marcas temporales, MVCC).
- **Durabilidad (D).** Una vez que una transacción confirmó (COMMIT), sus cambios persisten en disco aun ante fallos posteriores. Lo asegura el sistema de logs + flushes.

3. Estados de una transacción

Durante su vida, una transacción pasa por una secuencia de estados:

- **Activa:** está ejecutándose normalmente.
- **Parcialmente comprometida:** ejecutó su última instrucción pero todavía hay trabajo del motor pendiente (escribir log, propagar cambios).
- **Fallida:** algo impide continuar la ejecución (error en la aplicación, violación de constraint, deadlock, fallo del sistema).
- **Abortada:** se hizo rollback y la base se restauró al estado previo. Una transacción abortada se puede *reintentar* (si el error fue transitorio, como un deadlock) o *cancelar definitivamente*.

- **Comprometida:** se completó con éxito; sus efectos son permanentes.



4. Ejecución concurrente y planificaciones

Permitir que muchas transacciones corran al mismo tiempo trae beneficios concretos:

- **Mejor uso del hardware:** mientras una transacción usa CPU, otra puede estar leyendo disco.
- **Menor tiempo de respuesta promedio:** transacciones cortas no tienen que esperar detrás de transacciones largas.

Pero también introduce problemas: si las transacciones se entrelazan sin control, pueden violar la consistencia. Por eso hace falta un mecanismo de **control de concurrencia** que regule cómo se intercalan.

4.1. Planificación (*schedule*)

Una **planificación** es una secuencia que muestra en qué orden se ejecutan las operaciones de un conjunto de transacciones concurrentes. Tiene dos requisitos:

- Contiene todas las operaciones de todas las transacciones.
- Preserva el orden interno de cada transacción.

Planificación serial. Ejecuta cada transacción completa antes de empezar la siguiente; no hay intercalado. Son obviamente correctas porque cada transacción ve la base entera entre las dos consistentes. La idea de la concurrencia es *aproximarse* a una ejecución serial en términos de resultado, pero permitiendo intercalado para ganar rendimiento.

5. Serializabilidad

Asumimos dos cosas:

- Cada transacción tomada aisladamente preserva la consistencia de la base.
- Cualquier planificación serial preserva la consistencia.

Una planificación (posiblemente concurrente) es **secuenciable** (o *serializable*) si es *equivalente* a alguna planificación serial. Hay dos definiciones de equivalencia:

- Equivalencia **por conflicto** (la más usada y la del Práctico 6).
- Equivalencia **por vistas** (más permisiva pero NP-hard).

5.1. Conflicto entre operaciones

Dos instrucciones I_i de T_i y I_j de T_j están en conflicto si acceden al mismo ítem Q y al menos una de las dos es una escritura:

I_i	I_j	¿Conflicto?
$r(Q)$	$r(Q)$	No
$r(Q)$	$w(Q)$	Sí
$w(Q)$	$r(Q)$	Sí
$w(Q)$	$w(Q)$	Sí

5.2. Equivalencia por conflicto

Dos planificaciones son **equivalentes por conflicto** si una se puede transformar en la otra intercambiando *únicamente* instrucciones consecutivas que no estén en conflicto. Una planificación es **serializable por conflicto** si es equivalente por conflicto a alguna planificación serial.

5.3. Test del grafo de precedencia

El test algorítmico de serializabilidad por conflicto construye un grafo dirigido:

- Un nodo por cada transacción de la planificación.
- Un arco $T_i \rightarrow T_j$ por cada par de operaciones en conflicto donde la de T_i aparece *antes* que la de T_j .

La planificación es serializable por conflicto si y sólo si el grafo de precedencia es **acíclico**. Si hay un ciclo, ningún orden serial es compatible con todos los arcos a la vez, y por lo tanto no hay una serie equivalente.

Ejemplo (Práctico 6, Ejercicio 1). La planificación $r_1(X), r_2(X), w_1(X), w_2(X), \dots$ genera $T_1 \rightarrow T_2$ (por $r_1(X)$ antes de $w_2(X)$) y $T_2 \rightarrow T_1$ (por $r_2(X)$ antes de $w_1(X)$): hay ciclo, y la planificación no es serializable.

6. Recuperabilidad

La serializabilidad asegura que el resultado de la concurrencia “equivale” a una ejecución serial. Pero falta otra dimensión: ¿qué pasa si una transacción aborta? Para que se pueda hacer **ROLLBACK** correctamente, la planificación tiene que cumplir condiciones extra.

6.1. Planificación recuperable

Una planificación es **recuperable** si, para todo par T_i, T_j tal que T_j lee un dato que T_i escribió antes, el **commit** de T_i aparece antes que el de T_j . Si no se cumple, T_j podría haber confirmado basándose en una escritura que después se aborta: *no es posible deshacer eso*.

6.2. Planificación sin cascada

Una planificación es **sin cascada** (*cascadeless*) si para todo T_i, T_j tal que T_j lee un dato escrito por T_i , el **commit** de T_i ocurre antes que la *lectura* de T_j .

- Toda planificación sin cascada es recuperable.
- Lo recíproco no se cumple: hay recuperables que sí tienen cascada.

El **rollback en cascada** (que una falla obligue a abortar muchas transacciones encadenadas) es indeseable porque desperdicia trabajo y puede crear avalanchas. SQL exige que el sistema garantice tanto secuencialidad como ausencia de cascada.

7. Niveles de aislamiento (SQL 92)

Garantizar *serializabilidad estricta* para todas las transacciones es caro (más bloqueos, menos concurrencia). Para algunas aplicaciones —típicamente reportes y estadísticas— se acepta perder precisión a cambio de rendimiento. SQL 92 define cuatro niveles de aislamiento ordenados de más débil a más fuerte:

Nivel	Lectura sucia	No repetible	Phantom
READ UNCOMMITTED	Posible	Posible	Posible
READ COMMITTED	No posible	Posible	Posible
REPEATABLE READ	No posible	No posible	Posible (raro)
SERIALIZABLE	No posible	No posible	No posible

Los tres fenómenos.

- **Lectura sucia (*dirty read*):** T_1 lee datos que T_2 todavía no confirmó. Si T_2 aborta, T_1 leyó un fantasma.
- **Lectura no repetible:** T_1 lee la misma fila dos veces y obtiene valores distintos porque entre lectura y lectura otra transacción la actualizó y confirmó.
- **Phantom read:** T_1 ejecuta dos veces la misma consulta de rango y aparecen filas nuevas porque otra transacción insertó registros que cumplen la condición.

Comandos SQL.

```
-- MySQL
SET SESSION TRANSACTION ISOLATION LEVEL SERIALIZABLE;

-- PostgreSQL
SET DEFAULT_TRANSACTION_ISOLATION = 'serializable';
```

8. Control de concurrencia: protocolos basados en bloqueos

Los protocolos de bloqueo garantizan aislamiento haciendo que cada operación pase por un *lock manager* antes de poder ejecutarse.

8.1. Modos de bloqueo

- **Compartido (S/C):** lock-S(Q). El dato Q puede ser leído. Varias transacciones pueden tener S sobre Q a la vez.
- **Exclusivo (X):** lock-X(Q). El dato Q puede ser leído y escrito. Sólo una transacción puede tenerlo a la vez.

Matriz de compatibilidad:

	S	X
S	compatible	no compatible
X	no compatible	no compatible

Si una transacción T_i pide un bloqueo que choca con uno ya tomado, T_i **espera** hasta que se libere.

Bloquear no alcanza por sí solo. Si una transacción pide y suelta locks alegremente, distintos ítems pueden ser leídos de versiones inconsistentes entre sí, y el resultado no equivale a ninguna ejecución serial. Se necesita un *protocolo* que diga cuándo se pueden tomar y liberar.

8.2. Posibilidad de Deadlock

Si T_1 tiene $X(B)$ y pide $X(A)$, y T_2 tiene $S(A)$ y pide $S(B)$ (o $X(B)$), ninguna avanza. Esto es un **deadlock**. Para desbloquear, el motor debe hacer rollback de al menos una de las dos (la *víctima*); sus locks se liberan y la otra continúa.

9. Protocolo de bloqueo de dos fases (2PL)

El protocolo más usado para garantizar serializabilidad por conflicto mediante bloqueos es el **Bloqueo de Dos Fases**:

- **Fase de crecimiento:** la transacción puede solicitar bloqueos. No puede liberar ninguno.
- **Fase de decrecimiento:** la transacción puede liberar bloqueos. No puede solicitar ninguno nuevo.

El punto donde una transacción libera su primer lock se llama *punto de bloqueo*. A partir de ahí, todo lo que puede hacer es operar con lo que ya tiene y soltar locks.

Garantía. Toda planificación legal bajo 2PL es serializable por conflicto. La prueba se basa en mostrar que el orden de los puntos de bloqueo de las transacciones induce un orden serial equivalente.

9.1. Variantes

- **2PL puro o básico:** el descrito arriba. Permite rollback en cascada (porque una transacción puede haber leído de otra que después aborta).
- **2PL estricto:** todos los *bloqueos exclusivos* se mantienen hasta el final (commit/rollback). Evita el rollback en cascada.
- **2PL riguroso:** *todos* los bloqueos (S y X) se retienen hasta el final. Asegura sin cascada y, además, las transacciones se serializan en el orden de sus commits. Es la variante que implementan la mayoría de los motores comerciales.

9.2. Conversión de bloqueos

Para ganar concurrencia, 2PL puede admitir *conversiones*:

- Subir: de S a X (sólo durante la fase de crecimiento).
- Bajar: de X a S (sólo durante la fase de decrecimiento).

Si los locks se retienen hasta el final, las conversiones son seguras y las planificaciones quedan sin cascada.

10. Protocolos basados en grafos

Otra alternativa al 2PL son los protocolos basados en grafos. Imponen un orden parcial $\rightarrow$ sobre los ítems de datos: si $d_i \rightarrow d_j$, toda transacción que use ambos debe bloquear d_i *antes* de d_j .

10.1. Protocolo de árbol

El conjunto de ítems se organiza como un árbol enraizado. Sólo permite bloqueos exclusivos. Reglas:

1. El *primer* bloqueo de T_i puede ser sobre cualquier ítem.
2. Los siguientes ítems sólo pueden bloquearse si T_i tiene actualmente bloqueado el padre.
3. Los ítems se pueden desbloquear en cualquier momento.
4. T_i no puede volver a bloquear un ítem que ya bloqueó y desbloqueó.

Ventajas frente a 2PL.

- No hay deadlocks (porque el orden está fijo de antemano).
- Las transacciones pueden liberar locks tempranamente, ganando concurrencia.

Desventajas.

- Una transacción puede tener que bloquear ítems que no necesita, sólo para llegar a los que sí (*lock pre-emption*).
- Las planificaciones pueden no ser recuperables ni sin cascada.

11. Protocolo de ordenación por marcas temporales

La alternativa a los bloqueos es decidir *a priori* el orden serial equivalente y forzar al schedule a respetarlo. Para eso cada transacción T_i recibe al iniciar una **marca temporal** $MT(T_i)$ única (reloj del sistema o contador). Si $MT(T_i) < MT(T_j)$, el motor se compromete a producir un schedule equivalente a la serie en la que T_i va antes que T_j .

Información que mantiene cada ítem.

- $MT-E(Q)$: mayor marca de las transacciones que escribieron Q con éxito.
- $MT-L(Q)$: mayor marca de las transacciones que leyeron Q con éxito.

11.1. Reglas del protocolo

T_i ejecuta $read(Q)$.

- Si $MT(T_i) < MT-E(Q)$: T_i querría leer una versión que ya fue sobrescrita por una transacción más nueva. La lectura se rechaza y T_i se retrocede.
- En otro caso: se ejecuta y $MT-L(Q) \leftarrow \max(MT-L(Q), MT(T_i))$.

T_i ejecuta $write(Q)$.

- Si $MT(T_i) < MT-L(Q)$: el valor que T_i produciría ya fue necesario por una lectura posterior; la escritura llega “tarde” y T_i se retrocede.
- Si $MT(T_i) < MT-E(Q)$: T_i querría sobrescribir con un valor obsoleto; se retrocede.
- En otro caso: se ejecuta y $MT-E(Q) \leftarrow MT(T_i)$.

11.2. Propiedades

- Garantiza planificaciones serializables.
- No tiene deadlocks (las transacciones nunca esperan; se retroceden).
- Puede generar planificaciones *no recuperables*. Para asegurar recuperabilidad hay que extenderlo (por ejemplo, retrasar el `commit` de T_i hasta que todas las que escribieron datos leídos por T_i hayan commitado).
- Costo: las transacciones largas suelen ser retrocedidas y reiniciadas (*starvation*).

12. Esquemas multiversión y MVCC

Los esquemas **multiversión** mantienen varias versiones de cada ítem de datos para incrementar la concurrencia. Cuando una transacción lee, el control de concurrencia le entrega la versión adecuada según su marca temporal o su snapshot, sin tener que bloquear escritores.

12.1. Ordenación por marcas temporales multiversión

Cada ítem Q tiene una secuencia de versiones $Q_1, Q_2, \dots, Q_m$. Cada versión Q_k guarda:

- **contenido**: el valor de la versión.
- $MT-E(Q_k)$: la marca temporal de la transacción que creó esa versión.
- $MT-L(Q_k)$: la mayor marca temporal entre las transacciones que leyeron esa versión.

Cuando T_i pide $read(Q)$, el motor elige la versión Q_k con $MT-E(Q_k)$ máximo que sea $\leq MT(T_i)$: “la versión más reciente que ya existía cuando empezó T_i ”. Para $write(Q)$, si $MT(T_i) < MT-L(Q_k)$ T_i se retrocede; si $MT(T_i) = MT-L(Q_k)$ se sobrescribe esa versión; en otro caso se crea una nueva.

Ventaja clave. Las lecturas *nunca esperan*: hay una versión apropiada y se devuelve directamente.

12.2. MVCC en PostgreSQL

PostgreSQL implementa **MVCC** (*Multi-Version Concurrency Control*) y agrega a cada tupla dos columnas de sistema:

- **xmin**: el XID de la transacción que *insertó* esa versión.
- **xmax**: el XID de la transacción que la *invalida* (por update o delete); 0 si la versión sigue viva.

Cada `UPDATE` no machaca la tupla: crea una versión nueva con **xmin** = XID actual y deja a la versión vieja marcada con **xmax** = XID actual. Cuando una transacción quiere leer la fila, elige la versión que sea visible para su snapshot.

Consecuencia operativa.

- Lecturas no bloquean escrituras, y viceversa.
- Dos transacciones concurrentes pueden ver versiones distintas de la misma fila simultáneamente (ejercicio 8 del Práctico 6).
- Las versiones muertas se eliminan con `VACUUM` o el `autovacuum`.

PostgreSQL combina MVCC con bloqueos explícitos cuando la operación lo requiere: `SELECT ... FOR UPDATE`, `LOCK TABLE`, etc.

12.3. Modos de bloqueo de tabla en PostgreSQL

PostgreSQL ofrece varios modos de bloqueo a nivel tabla, en general adquiridos automáticamente por el comando:

Modo	Adquirido por
ACCESS SHARE	SELECT
ROW SHARE	SELECT FOR UPDATE/SHARE
ROW EXCLUSIVE	INSERT, UPDATE, DELETE
SHARE UPDATE EXCLUSIVE	VACUUM, autoanalyze
SHARE	CREATE INDEX
SHARE ROW EXCLUSIVE	— (manual)
EXCLUSIVE	— (manual)
ACCESS EXCLUSIVE	ALTER/DROP TABLE, TRUNCATE, REINDEX

Excepto ACCESS SHARE, los modos adquiridos en una transacción se retienen hasta el final.

13. Fenómeno fantasma e índices

El control de concurrencia con *granularidad de tupla* (locks por fila, o marcas por fila) no detecta conflictos *sobre tuplas todavía no creadas*.

Ejemplo. T_1 ejecuta

```
SELECT min(precio) FROM provee WHERE nart = 2;
```

y T_2 ejecuta

```
INSERT INTO provee (nart, nprov, precio) VALUES (2, 4, 1.23);
```

No hay tupla común; con granularidad de tupla el sistema no ve conflicto. Sin embargo, T_1 podría ver el INSERT de T_2 en una segunda ejecución y ya no sería repetible. Este problema se llama **fenómeno fantasma**.

Solución. Bloquear *a nivel de predicado o de índice*: un lock sobre `provee.nart = 2` impide a otras transacciones insertar filas que cumplan esa condición. Los motores reales lo implementan vía *predicate locks* o *index range locks*.

14. Transacciones en SQL

Una transacción SQL queda iniciada implícitamente (al ejecutar la primera sentencia) o explícitamente (BEGIN / START TRANSACTION) y termina con:

- COMMIT: confirma todas las escrituras realizadas.
- ROLLBACK: las descarta y restaura el estado anterior.

Lo que la norma exige. El sistema debe garantizar *secuencialidad* (serializabilidad por conflicto) y ausencia de rollback en cascada. Implementaciones reales suelen relajar la serializabilidad estricta y dejar al programador elegir el nivel de aislamiento.

Comandos prácticos.

```
-- MySQL
SET autocommit = 0;
START TRANSACTION;
... -- operaciones
COMMIT; -- o ROLLBACK

-- PostgreSQL
BEGIN;
... -- operaciones
COMMIT; -- o ROLLBACK
```

Constraints diferidas (PostgreSQL). La cláusula `DEFERRABLE INITIALLY DEFERRED` en una FK posterga la verificación hasta el `COMMIT`, permitiendo estados intermedios “inconsistentes” dentro de la transacción. Es útil cuando dos `INSERT` se referencian mutuamente.

15. Cómo se conecta con el Práctico 6

Los conceptos del Teórico 8 aparecen en el Práctico 6 distribuidos así:

- **Ejercicio 1:** serializabilidad por conflicto + grafo de precedencia (sección sobre serializabilidad).
- **Ejercicio 2:** protocolo de bloqueo de dos fases puro (sección sobre 2PL).
- **Ejercicio 3:** protocolo de ordenación por marcas temporales (sección sobre marcas temporales).
- **Ejercicio 4:** atomicidad y `ROLLBACK` en MySQL (sección sobre transacciones en SQL).
- **Ejercicio 5:** nivel `SERIALIZABLE` y bloqueos en MySQL (sección sobre niveles de aislamiento y bloqueos).
- **Ejercicio 6:** lectura no repetible y niveles `READ COMMITTED` / `REPEATABLE READ` (sección sobre niveles de aislamiento).
- **Ejercicio 7:** constraints deferibles en PostgreSQL (sección sobre transacciones en SQL).
- **Ejercicio 8:** MVCC, `xmin`, `xmax` (sección sobre esquemas multiversión).

16. Conclusión

Las ideas centrales a retener:

- Una transacción es la unidad lógica del trabajo concurrente; el motor garantiza para ella las propiedades ACID.
- El control de concurrencia tiene que asegurar *aislamiento*: el resultado de la ejecución intercalada debe ser equivalente a alguna ejecución serial.
- El test del grafo de precedencia es la herramienta formal para decidir si un schedule es serializable por conflicto.
- Hay dos grandes familias de protocolos: basados en bloqueos (2PL y variantes) y basados en marcas temporales. Los motores modernos suelen combinar ambos y agregar multiversión (MVCC).
- Los niveles de aislamiento (`READ UNCOMMITTED`, `READ COMMITTED`, `REPEATABLE READ`, `SERIALIZABLE`) ofrecen un trade-off explícito entre consistencia y concurrencia, regulando qué fenómenos pueden aparecer.
- Recuperabilidad y ausencia de rollback en cascada son propiedades adicionales que la planificación debe cumplir para que el sistema pueda revertir transacciones de manera

limpia.